



Universidad Nacional Autónoma de México



Facultad de Ingeniería

Práctica 2: CONTROL DE LEDS.

Integrantes:

Espinoza Matamoros Percival Ulises - 320025561

Flores Colin Victor Jaziel - 320266083

García Cortés Adolfo de Jesús - 320317264

Lara Hernandez Angel Husiel - 320060829

Lugo Manzano Rodrigo - 320206968

Microcomputadoras

Grupo: 03 - Semestre: 2026-2

Calf: _____



1. Objetivo:

Mediante la creacion, implementacion de un programa en MicroPython, diseñar el control de 8 leds conectados a una Raspberry Pi Pico, utilizando diferentes secuencias de encendido y apagado, asi como la capacidad de tener la lectura desde la terminal para saber las entradas que se estaran realizando para cada secuencia, asi mismo contruir fisicamente el circuito con los leds y las resistencias necesarias para su correcto funcionamiento.

2. Desarrollo:

2.1. Lógica de Control a Nivel de Bits

Para optimizar el manejo de los pines de salida de la Raspberry Pi Pico sin tener que declarar el estado de cada pin de forma individual y repetitiva, se implementó la función `set_leds(valor)`, esta función recibe como argumento un número entero el cual representa el estado de los 8 LEDs y se encarga de traducirlo a señales físicas en el microcontrolador.

El funcionamiento de esta parte se basa en operadores de corrimiento a nivel de bits. Dentro de un ciclo `for` que itera 8 veces, una por cada LED, se aplica la instrucción `(valor >> i) & 1`.

Primero, el operador de corrimiento a la derecha (`>>`) desplaza los bits del `valor` ingresado `i` posiciones hacia la derecha. Esto coloca el bit específico que se desea evaluar en la posición del bit menos significativo. Luego, se aplica la operación lógica AND (`&`) con el número 1, esta operación actúa como un filtro para el resto de los bits y separa únicamente el menos significativo, devolviendo como resultado final un 1 lógico (encendido) o un 0 lógico (apagado). Y ya por último, este resultado se envía al método `.value()` del pin correspondiente.

Listing 1: Función para manejar el estado de los pines mediante lógica a nivel de bits

```
1 def set_leds(valor):  
2     for i in range(8):  
3         pines_leds[i].value((valor >> i) & 1)
```



2.2. Implementación de un menú de usuario

Para facilitar la interacción del usuario con el programa, se hizo la función `mostrar_menu()`, con la finalidad de imprimir en la consola de manera estructurada las 8 combinaciones binarias posibles y la acción correspondiente a cada una, mejorando la usabilidad general. Además, esta función se mandará a llamar más tarde y así también mejoramos la legibilidad del código al separar la lógica de presentación de la lógica de control.

Listing 2: Interfaz de menú y función de lectura en hilo secundario

```
1 def mostrar_menu():
2     print("\n" + "="*60)
3     print(" " * 18 + "CONTROL DE LEDs")
4     print("="*60)
5     print(" [000] - Todos los LEDs apagados")
6     print(" [001] - Encender / Apagar LEDs")
7     print(" [010] - Encender / Apagar Nibbles")
8     print(" [011] - Corrimientos entre Nibbles a la derecha")
9     print(" [100] - Corrimientos entre Nibbles a la izquierda")
10    print(" [101] - Corrimientos de LEDs a la derecha")
11    print(" [110] - Corrimientos de LEDs a la izquierda")
12    print(" [111] - Corrimientos de LEDs en Zig - Zag")
13    print("="*60)
```

2.3. Implementación de Multihilo

Uno de los principales retos en el diseño del programa para este sistema de control fue lograr que las secuencias luminosas se ejecutaran de manera ininterrumpida mientras el programa se mantenía a la espera de nuevas instrucciones. En una ejecución secuencial clásica, invocar la función `input()` detiene por completo el flujo del programa hasta que el nosotros como usuarios ingresamos un dato por teclado, lo cual congelaría el parpadeo de los LEDs.

Para resolver esta parte, implementamos el uso de la biblioteca `_thread` nativa de MicroPython, esta biblioteca nos permite crear un hilo de ejecución secundario que opera en paralelo al procesamiento principal del microcontrolador.

La captura de datos se implemento en la función `leer_entradas()`, que es la que se ejecuta en el hilo secundario, despliega el menú y entra en un ciclo infinito evaluando la entra-



da por teclado. Al recibir una cadena, verifica que pertenezca al arreglo de combinaciones válidas (desde '000' hasta '111') y de ser correcta, actualiza de inmediato la variable global `Decision`. Al utilizar `_thread.start_new_thread(Leer_entradas, ())`, el hilo principal queda libre de interrupciones, dedicándose exclusivamente a leer la variable `Decision` y ejecutar las secuencias de tiempo crítico en el hardware.

Listing 3: Interfaz de menú y función de lectura en hilo secundario

```
1 def leer_entradas():
2     global Decision
3     Entradas = ['000', '001', '010', '011', '100', '101', '110', '
4         111']
5
6     mostrar_menu()
7     print(f'\nESTADO INICIAL: [{Decision}] - Todos los LEDs apagados
8         ')
9
10    while True:
11        nueva_opcion = input('\n>> Teclee la combinacion [PIN11,
12            PIN15, PIN17]: ')
13
14        if nueva_opcion in Entradas:
15            Decision = nueva_opcion
16
17            print("\n" + "-"*60)
18            if Decision == '000':
19                print(f'    Ejecutando: [{Decision}] - Todos los LEDs
20                    apagados')
21            elif Decision == '001':
22                print(f'    Ejecutando: [{Decision}] - Encender /
23                    Apagar LEDs')
24            elif Decision == '010':
25                print(f'    Ejecutando: [{Decision}] - Encender /
26                    Apagar Nibbles')
27            elif Decision == '011':
28                print(f'    Ejecutando: [{Decision}] - Corrimientos
29                    entre Nibbles (Derecha)')
```



```
23         elif Decision == '100':
24             print(f'    Ejecutando: [{Decision}] - Corrimientos
                entre Nibbles (Izquierda)')
25         elif Decision == '101':
26             print(f'    Ejecutando: [{Decision}] - Corrimientos
                de LEDs (Derecha)')
27         elif Decision == '110':
28             print(f'    Ejecutando: [{Decision}] - Corrimientos
                de LEDs (Izquierda)')
29         elif Decision == '111':
30             print(f'    Ejecutando: [{Decision}] - Corrimientos
                de LEDs en Zig-Zag')
31     print("-" * 60)
32     else:
33         print("\n" + "!"*60)
34         print("    Opcion no valida.")
35         print("    Entradas disponibles: 000, 001, 010, 011, 100,
                101, 110, 111")
36         print("!"*60)
```

2.4. Diseño y Lógica Combinacional de las Secuencias

El hilo principal del programa está formado por un ciclo infinito (`while True`) que evalúa de forma continua la variable global `Decision`. Dependiendo de su estado lógico, el cual representa las combinaciones binarias desde '000' hasta '111', el flujo de ejecución se direcciona hacia rutinas específicas. Para optimizar el procesamiento en la Raspberry Pi Pico, estas rutinas se agruparon en tres categorías:

2.4.1. Secuencias de estado fijo y alternancia de Nibbles (000, 001 y 010)

Para los comportamientos más directos, como apagar todos los LEDs (000), generar un parpadeo general (001) o el parpadeo alternado de Nibbles (010), se enviaron directamente literales binarios a la función `set_leds`. Por ejemplo, para los Nibbles se alternaron las máscaras `0b11110000` (parte alta) y `0b00001111` (parte baja), controlando la frecuencia de oscilación con la instrucción `time.sleep(0.2)`.



2.4.2. Corrimientos direccionales iterativos (011, 100, 101 y 110)

Los casos que implican desplazamientos espaciales de LEDs o Nibbles a lo largo de la barra se implementaron utilizando listas que contienen los estados precalculados en formato binario y un ciclo `for` itera sobre estos arreglos y envía el estado correspondiente.

Un detalle importante en esta implementación es el manejo del control asíncrono, pues dentro de cada iteración del ciclo `for`, se agregó una validación de estado (`if Decision != '...': break`). Esto nos garantiza que, si el hilo secundario modifica la variable de control ante un nuevo comando del usuario, el ciclo de corrimiento en curso se interrumpa instantáneamente, logrando un tiempo de respuesta inmediato en el hardware y evitando que la secuencia termine de ejecutarse antes de aplicar el cambio.

2.4.3. Implementación matemática del corrimiento en Zig-Zag (111)

Para lograr el efecto de rebote de luces tipo Zig-Zag, y no tener que definir una extensa lista de estados de 8 bits, se usó un arreglo que representa únicamente las posiciones espaciales: `secuencia = [0, 1, 2, 3, 4, 5, 6, 7, 6, 5, 4, 3, 2, 1]`.

Haciendo uso de un contador global que se incrementa infinitamente (`paso`) y el operador de módulo (%), se calcula un índice dinámico pero seguro mediante la expresión `paso % len(secuencia)`. Una vez obtenida la posición de la iteración actual, se emplea un operador de corrimiento lógico a la izquierda (`1 << posicion`) para colocar un único bit en estado alto exactamente en el pin físico que le corresponde.

Listing 4: Lógica de control del ciclo principal y casos principales de cada categoría

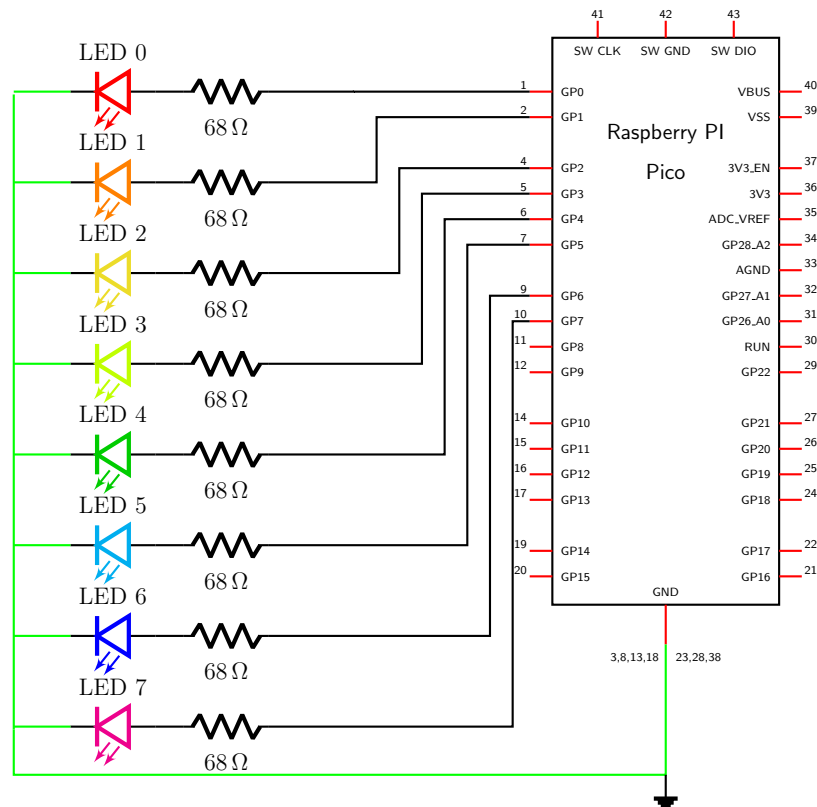
```
1  while True:
2      if Decision == '000':
3          set_leds(0b00000000)
4          time.sleep(0.1)
5          # ... Casos 001 a 100
6
7      elif Decision == '101':
8          secuencia = [0b10000000, 0b01000000, 0b00100000, 0
9                      b00010000, 0b00001000, 0b00000100, 0b00000010, 0
                        b00000001]
10         for valor in secuencia:
```



```
10         set_leds(valor)
11         time.sleep(0.2)
12         if Decision != '101':#Condicion de ruptura
13             break
14     # ... Caso 110
15
16     elif Decision == '111':
17         secuencia = [0, 1, 2, 3, 4, 5, 6, 7, 6, 5, 4, 3, 2, 1]
18         posicion = secuencia[paso % len(secuencia)]
19         set_leds(1 << posicion)
20         paso += 1
21         time.sleep(0.2)
```

2.5. Implementación Física del Circuito

Para la implementacion de este proyecto se utiliza la Raspberry Pi Pico, cuyo diagrama de pines se muestra a continuacion:





Como se puede observar en el diagrama, el hardware se estructuró usando los pines de propósito general GPIO de la Raspberry. Se seleccionaron 8 salidas digitales consecutivas lógicamente (GP0, GP1, GP2, GP3, GP4, GP5, GP6 y GP7) para mantener un orden en el bus de datos de la función `set_leds()`.

Para proteger tanto los puertos del microcontrolador como el display multicolor, se colocó una resistencia limitadora de corriente de 68Ω en serie con cada una de las líneas de señal.

También optamos por una mejora en el diseño físico, ya que en lugar de alambrar 8 LEDs individuales, se empleó un display tipo barra multicolor de 10 segmentos, de esta barra, se conectaron y utilizaron únicamente 8 segmentos que corresponden a nuestros 8 bits de control. Esta decisión de hardware nos resultó ser mucho más funcional, primero en la implementación en la protoboard y además nos permitió una presentación mucho más limpia visualmente.



3. Resultados:



4. Conclusiones:

- Espinoza Matamoros Percival Ulises:
- Flores Colin Victor Jaziel:
- Garcia Cortes Adolfo de Jesus:
- Lara Hernandez Angel Husiel:
- Lugo Manzano Rodrigo:



Referencias

- Cárdenas, H., Riera, E. L., & Raggi, F. (1990). *Álgebra superior* (2.^a ed.). Editorial Trillas.
- Chapra, S. C., & Canale, R. P. (2007). *Métodos numéricos para ingenieros* (5.^a ed.). McGraw-Hill.
- Ivorra, C. (s.f.). *Las fórmulas de Cardano-Ferrari*. <https://www.uv.es/~ivorra/Libros/Ecuaciones.pdf>
- Nieuwpoort, R. (2026). *Cardano's formula*. <https://rubenvannieuwpoort.nl/posts/cardanos-formula>
- Python Software Foundation. (2026a). *cmath — Mathematical functions for complex numbers*. <https://docs.python.org/3/library/cmath.html>
- Python Software Foundation. (2026b). *math — Mathematical functions*. <https://docs.python.org/3/library/math.html>
- Raspberry Pi Ltd. (2025a). *Raspberry Pi Pico 2 W Datasheet*. <https://pip-assets.raspberrypi.com/categories/1088-raspberry-pi-pico-2-w/documents/RP-008304-DS-2-pico-2-w-datasheet.pdf?disposition=inline>
- Raspberry Pi Ltd. (2025b, 20 de febrero). *RP2040 Datasheet: A microcontroller by Raspberry Pi*. Ver. build-version: 3184e62-clean. <https://pip-assets.raspberrypi.com/categories/814-rp2040/documents/RP-008371-DS-1-rp2040-datasheet.pdf>